



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №1

з дисципліни «Технології розроблення програмного забезпечення»

Тема: «Git»

Виконав:

студент групи ІА-32
Шадлер Андрій

Перевірив:

вик. Мягкий М. Ю.

Зміст

Теоретичні відомості.....	2
Системи контролю версій.....	2
Git – розподілена система контролю версій	2
Основні команди Git.....	3
Конфлікти при злитті.....	3
Хід виконання:.....	3
Висновок	5

Теоретичні відомості

Системи контролю версій

Система контролю версій (СКВ) – це програмне забезпечення, яке дозволяє відстежувати зміни у файлах, зберігати історію їх розвитку та організовувати командну роботу над проєктами.

Види СКВ:

- **Централізовані (CVS, Subversion)** – мають єдиний сервер із репозиторієм, усі користувачі працюють напямую з ним.
- **Розподілені (Git, Mercurial)** – кожен розробник має повну копію репозиторію на своєму комп'ютері, що робить роботу швидшою та більш надійною.

Git – розподілена система контролю версій

Git – найпоширеніша сучасна СКВ. Вона дозволяє:

- створювати **репозиторій** для зберігання історії проєкту;
- фіксувати зміни у вигляді **комітів**;
- працювати з кількома **гілками** (branches) для розділення різних напрямків розробки;
- об'єднувати гілки за допомогою **merge**;
- вирішувати **конфлікти**, коли в різних гілках змінено один і той самий фрагмент файлу.

Основні команди Git

- `git init` – створення нового локального репозиторію.
- `git add <файл>` – додавання файлів до індексу (staging area).
- `git commit -m "message"` – створення коміту з описом змін.
- `git branch` – перегляд гілок.
- `git branch <name>` / `git checkout -b <name>` – створення нової гілки.
- `git checkout <name>` або `git switch <name>` – перехід до іншої гілки.
- `git merge <name>` – злиття в поточну гілку змін з іншої.
- `git status` – перевірка стану файлів і репозиторію.

Конфлікти при злитті

Конфлікт виникає, якщо однакові рядки одного файлу були змінені у двох різних гілках. Git не може автоматично вирішити, яку версію залишити.

Щоб вирішити конфлікт:

1. Відкрити файл і залишити потрібний варіант (або об'єднати обидва).
2. Видалити службові позначки (<<<<<<, =====, >>>>>>).
3. Виконати `git merge --continue`

Хід виконання:

1. Створення та ініціалізація Git репозиторію
2. Створення пустого файлу `file.txt`
3. Додавання першого коміту

```
PS B:\Study\КРІ\3 Курс\1 Семестр\ТРПЗ\lab1> git init
Initialized empty Git repository in B:/Study/КРІ/3 Курс/1 Семестр/ТРПЗ/lab1/.git/
PS B:\Study\КРІ\3 Курс\1 Семестр\ТРПЗ\lab1> git add .
PS B:\Study\КРІ\3 Курс\1 Семестр\ТРПЗ\lab1> git commit -m "init commit"
[master (root-commit) 2274bd9] init commit
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 file.txt
```

4. Створення двох додаткових гілок
5. Виведення загального списку гілок

```

PS B:\Study\KPI\3 Курс\1 Семестр\ТПЗ\lab1> git branch new-branch
PS B:\Study\KPI\3 Курс\1 Семестр\ТПЗ\lab1> git checkout -b another-branch
Switched to a new branch 'another-branch'
PS B:\Study\KPI\3 Курс\1 Семестр\ТПЗ\lab1> git branch
* another-branch
  master
  new-branch

```

6. Зміна тексту в файлі file.txt на “changes”
7. Перевірка статусу гілки
8. Створення першого коміту на відповідну гілку

```

PS B:\Study\KPI\3 Курс\1 Семестр\ТПЗ\lab1> git status
On branch another-branch
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   file.txt

no changes added to commit (use "git add" and/or "git commit -a")
PS B:\Study\KPI\3 Курс\1 Семестр\ТПЗ\lab1> git add .
PS B:\Study\KPI\3 Курс\1 Семестр\ТПЗ\lab1> git status
On branch another-branch
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   file.txt

PS B:\Study\KPI\3 Курс\1 Семестр\ТПЗ\lab1> git commit -m "changed file"
[another-branch 7a7b0df] changed file
1 file changed, 1 insertion(+)

```

9. Перехід на гілку “new-branch”
10. Зміна тексту на “changes2”
11. Створення коміту

```

PS B:\Study\KPI\3 Курс\1 Семестр\ТПЗ\lab1> git checkout new-branch
Switched to branch 'new-branch'
PS B:\Study\KPI\3 Курс\1 Семестр\ТПЗ\lab1> git add .
PS B:\Study\KPI\3 Курс\1 Семестр\ТПЗ\lab1> git commit -m "changed file2"
[new-branch abee064] changed file2
1 file changed, 1 insertion(+)

```

12. Знаходячись у “new-branch” робимо злиття з гілкою “another-branch”

```

PS B:\Study\KPI\3 Курс\1 Семестр\ТПЗ\lab1> git merge another-branch
Auto-merging file.txt
CONFLICT (content): Merge conflict in file.txt
Automatic merge failed; fix conflicts and then commit the result.

```

```

<<<<<< HEAD
changes
=====
changes2
>>>>>> another-branch

```

13.Виправлення конфлікту вручну

14.Відновлення злиття командою git merge –continue

```
changes  
changes2
```

```
PS B:\Study\KPI\3 Курс\1 Семестр\ТПЗ\lab1> git add .  
PS B:\Study\KPI\3 Курс\1 Семестр\ТПЗ\lab1> git merge --continue  
[new-branch e942132] Merge branch 'another-branch' into new-branch
```

15.Виведення логу в підсумку виконання лабораторної роботи

```
PS B:\Study\KPI\3 Курс\1 Семестр\ТПЗ\lab1> git log --all --graph  
*   commit e94213274595f4aebffc95bc9f8588d8b626b690 (HEAD -> new-branch)  
|  \ Merge: abee064 7a7b0df  
|   Author: ShadlerAndrii <a.a.shadler@gmail.com>  
|   Date:   Mon Sep 22 08:55:22 2025 +0300  
|  
|       Merge branch 'another-branch' into new-branch  
|  
| *   commit 7a7b0df0c4e4ff0c372e160b78ba4b97053d756c (another-branch)  
| |  \ Author: ShadlerAndrii <a.a.shadler@gmail.com>  
| |   Date:   Mon Sep 22 08:50:05 2025 +0300  
| |  
| |      changed file  
| |  
| *   commit abee064b779ee8e9254afb2de8e0b8a7f25a662a  
| |  \ Author: ShadlerAndrii <a.a.shadler@gmail.com>  
| |   Date:   Mon Sep 22 08:51:37 2025 +0300  
| |  
| |      changed file2  
| |  
| *   commit 2274bd915100cd22bbbd25415580609f49b4efe9 (master)  
| |  \ Author: ShadlerAndrii <a.a.shadler@gmail.com>  
| |   Date:   Mon Sep 22 08:45:24 2025 +0300  
| |  
| |      init commit
```

Висновок

У ході лабораторної роботи було засвоєно основні принципи роботи з системою контролю версій **Git**. Було створено репозиторій, гілки, виконано коміти та злиття. На практиці відпрацьовано вирішення конфліктів і перегляд історії змін. Отримані навички дозволяють використовувати **Git** для організації командної роботи та керування версіями проєктів.